

DeadlineDesk

Digital Assignment & Deadline Manager

A student portfolio web app for tracking assignments and never missing a deadline

Presentation & Defense Study Guide

Student: AZET SIEMBE

Student Number (SIN): 2511159013

Built with HTML5, CSS3 and JavaScript

1. The Idea — What & Why

What problem does it solve?

Students juggle assignments from many subjects at once, each with its own due date. It is very easy to forget a deadline, underestimate how close it is, or lose track of what still needs to be done. DeadlineDesk solves this by giving a student one single place to add every assignment, see how urgent each one is, and check their overall progress at a glance.

Why is this a useful idea?

Missing a deadline can cost real marks, so a simple, free tool that "watches" every due date for you is genuinely useful. It does not need an account, a server, or an internet connection after the page loads — it just works, right in the browser, which makes it realistic for a student to actually use every day.

Motivation for choosing this project

- It solves a real, everyday problem that every student (including the developer) experiences.
- It is a great way to practice core front-end skills — HTML structure, CSS design, and JavaScript logic — without needing a backend server or database.
- It allows for visually interesting features (countdowns, risk colours, progress rings, a calendar) that make the project satisfying to build and demonstrate.

2. Technologies Used

The whole project is built with only three core web technologies, plus one small icon library. No frameworks, no backend, and no database were used — everything runs directly in the browser.

- **HTML5** — the skeleton of every page. It defines the structure: the navigation bar, headings, forms (like the "Add Assignment" form), buttons, and the footer. There are four HTML pages: `index.html` (Home), `assignments.html`, `calendar.html` and `dashboard.html`.
- **CSS3** — the styling. All colours, spacing, fonts and layout come from a custom design system built with CSS variables (also called "custom properties"). This is explained more in Section 7.
- **JavaScript** — the "brain" of the app. It handles everything interactive: adding/editing/deleting assignments, calculating how urgent a deadline is, building the calendar grid, updating countdown timers every second, and saving/loading data.
- **Browser localStorage** — a simple way the browser itself remembers data even after you close the tab or restart your computer — without needing a server or a database. Think of it as a small storage box built into the browser that belongs only to this website. DeadlineDesk saves all assignments and the light/dark theme choice into localStorage as a single piece of text (JSON).
- **Lucide Icons** — a small, free icon library loaded from the internet ([unpkg.com](https://lucide.dev)) that provides clean, consistent line icons (calendar, bell, checkmark, etc.) instead of using plain emoji, giving the app a more professional, modern look.

3. How the App Works (Page by Page)

Home page (index.html)

This is the landing page. It introduces the app with a hero section, explains the main features in a grid of cards, and shows a "How It Works" 3-step guide. It also shows live quick-stats (Total Assignments, Pending, Completion Rate) pulled straight from whatever is currently saved in localStorage, using home.js. These numbers count upward with a small animation when the page loads.

Assignments page (assignments.html)

This is the main workspace. Here the student can:

- Click "+ Add Assignment" to open a pop-up form (a "modal") and create a new assignment with a title, subject, priority, deadline date/time, and an optional description.
- Edit or delete any existing assignment.
- Tick a checkbox to mark an assignment as completed.
- Search assignments by keyword, and filter by subject, priority, or status (pending/completed/overdue).
- Sort the list by soonest deadline, latest deadline, priority, title, or most recently added.

Every assignment card shows a colour-coded "risk badge" and a live countdown timer, both explained in Section 5. This logic lives in js/assignments.js.

Calendar page (calendar.html)

This page draws a full monthly calendar grid (built entirely in JavaScript in js/calendar.js — there is no calendar library). Each day box shows any assignments due that day, colour-coded by urgency. Arrow buttons move between months, and a "Go to Today" button jumps back to the current month and highlights today's date.

Dashboard page (dashboard.html)

This is the "big picture" page. It shows four summary stat cards (Total, Completed, Pending, Overdue), a progress bar for each subject, a circular "Completion Rate" ring, a list of the soonest upcoming deadlines, and a list of urgent Deadline Alerts. This is built in js/dashboard.js and is the page most useful for showing overall progress in a presentation.

4. Key Features Explained Simply

1. Adding and tracking assignments by subject

Every assignment is a simple JavaScript object with a title, subject, priority, deadline, description, and a true/false "completed" flag. Subjects (Mathematics, Computer Science, English, Physics, Chemistry, History, Biology, Economics) are pre-defined in js/data.js, each with its own colour and icon, so tags look consistent everywhere in the app.

2. Deadline risk indicator (severity levels)

This is the app's signature feature. A function called `getRiskLevel()` checks how much time is left before the deadline and returns one word:

- Overdue — the deadline has already passed.
- Critical — less than 24 hours left.
- High — less than 3 days left.
- Medium — less than 7 days left.
- Low — more than 7 days left.
- Completed — the task is already marked done (shown in a neutral colour regardless of the date).

Each level has its own colour (from dark red for overdue to green for low risk), which is used consistently for badges, countdown text, and calendar dots, so the student can judge urgency at a glance without reading any numbers.

3. Live countdown timers

Every assignment card shows a countdown like "2d 14h" or "45m 12s". A JavaScript timer (`setInterval`) recalculates the remaining time once every second and updates the text on screen — this happens purely in the browser with no page reload.

4. Calendar view

The calendar is built by JavaScript, not a plug-in. The code works out how many days are in the chosen month and which weekday the 1st falls on, then generates one box per day, filling in any assignments due that day with a coloured dot matching their risk level.

5. Completion-rate dashboard and subject progress

The dashboard calculates statistics such as completion rate ($\text{completed} \div \text{total}$, as a percentage) and per-subject progress (how many tasks are done for Maths vs. English, etc.). These numbers are drawn as progress bars and an animated circular "ring" chart using basic SVG (a way to draw shapes and graphics directly in a web page).

6. Deadline alerts and dark/light theme toggle

The dashboard and assignments page list any assignment that is overdue, critical, or high risk as an "alert". On top of that, when the page loads and there are urgent items, a small pop-up notification ("toast") appears once per browser session to catch the student's attention. Separately, a sun/moon button in the navigation bar switches between light and dark colour themes instantly, and the chosen theme is remembered using `localStorage` so it stays the same next time the page is opened.

5. Code Structure Overview (Beginner-Friendly)

The project keeps a clean separation between structure (HTML), style (CSS) and behaviour (JavaScript). Inside the `js` folder, each file has one clear job:

- **css/styles.css** — every visual rule for the whole site: colours, fonts, spacing, card designs, the light/dark theme, and responsive rules for smaller screens. One file styles all four pages.
- **js/data.js** — the "content" file. It defines the list of subjects, priority levels, risk-level colours, and a set of sample assignments used the very first time the app runs (so the app is never empty on first visit).
- **js/common.js** — the shared toolbox, wrapped in something called the "DeadlineDesk" object. It holds everything used on every page: reading/writing localStorage, date and countdown formatting, calculating risk levels, adding/editing/deleting assignments, computing statistics, showing toast pop-ups, handling the theme toggle and mobile navigation menu, and refreshing icons. Every other JavaScript file calls into this one instead of repeating code.
- **js/home.js, js/assignments.js, js/calendar.js, js/dashboard.js** — one file per page. Each file only contains the logic specific to that page (for example, assignments.js manages the add/edit form and filters, while calendar.js builds the month grid). This keeps each file small and focused, which is a common, beginner-friendly way to organise a multi-page JavaScript project without using a framework.

6. Design Decisions — Why It Looks and Works This Way

- **Colour theme (indigo/blue accent):** a single accent colour (indigo, #4F46E5) is used consistently for buttons, links and highlights via a CSS variable, so the whole site feels like one coherent brand instead of a random mix of colours. Risk levels then use a separate red-to-green colour scale so urgency is instantly recognisable and never confused with the main brand colour.
- **Dark/light theme:** CSS variables such as `--bg` and `--text` are defined twice (once for light, once for dark) and swapped instantly when the `[data-theme]` attribute changes. This was included because many students study at night, and a dark mode is both a common expectation in modern apps and a lightweight, high-impact feature to add.
- **Mobile responsiveness:** the layout uses flexible grids and a collapsible hamburger menu (`navToggle`) so the same site works on a phone, tablet, or laptop without needing a separate mobile version. Students often check deadlines from their phones between classes, so this was an important, practical requirement, not just a nice-to-have.
- **Icons instead of emoji:** the Lucide icon library gives clean, uniformly-sized line icons that look the same across every device and browser. Emoji can render differently (or not at all) depending on the operating system, so real icons keep the design looking professional and consistent.
- **Working fully offline without a backend:** by saving everything in the browser's localStorage instead of a server/database, the app needs no hosting cost, no login system, and no internet connection to function after the page has loaded once. This matches the scope of a portfolio project — it proves the same front-end skills (HTML/CSS/JS) without the added complexity of managing a server.

7. Potential Presentation Questions & Answers

Q: Why did you use localStorage instead of a real database?

A: Because the project is a front-end-only web app with no server. localStorage lets the browser remember data for free, with no setup, no hosting costs, and no login — perfect for a portfolio project focused on demonstrating HTML, CSS and JavaScript skills.

Q: How do the deadline risk levels / severity get calculated?

A: A function compares the current time to the assignment's deadline and measures the number of hours remaining. Based on set thresholds (under 24h = Critical, under 72h = High, under 168h = Medium, more = Low, and past the deadline = Overdue) it returns a risk level, which then controls the badge colour and text everywhere in the app.

Q: What happens if the user clears their browser data or cache?

A: Since all data is stored in that browser's localStorage, clearing site data would remove all saved assignments, and the app would reset back to its starting sample assignments the next time it loads. This is a known trade-off of using localStorage instead of a server-based database.

Q: Why is there no backend or server?

A: The project's goal was to demonstrate core front-end web development skills. Adding a server and database would need a whole separate system (hosting, APIs, security) which was outside the scope of this assignment. localStorage lets the app still feel "real" and persistent without that complexity.

Q: What would you improve if you had more time?

A: I would add optional cloud sync/login so data could follow a student across devices, add recurring assignments and email/browser push reminders, and allow file attachments or grade tracking per assignment.

Q: What was the hardest part of building this?

A: Getting the calendar to correctly generate the right number of day boxes for any month (including leftover days from the previous/next month) and making sure the risk colours and countdown timers stayed perfectly in sync across every page was the trickiest logic to get right.

Q: How did you make the site work well on mobile phones?

A: The CSS uses flexible, responsive layouts and a collapsible hamburger navigation menu, so the same HTML adapts to smaller screens instead of needing a separate mobile version.

Q: What did you learn from building this project?

A: I learned how to structure a multi-page JavaScript project without a framework, how to design a consistent colour/theme system with CSS variables, how to persist data using localStorage, and how to break logic into small, reusable functions shared across pages.

Q: Why is the data split into separate files like data.js and common.js?

A: To keep the code organised and easy to maintain. data.js holds the static reference data (subjects, priorities, sample assignments), while common.js holds shared logic used by every page, so each page-specific file only needs to contain the code that is unique to that page.

Q: How does the app know which risk colour to show on the calendar?

A: The calendar looks up each day's assignments and calls the same getRiskLevel() function used everywhere else in the app, so the colour dot on the calendar always matches the badge shown on the Assignments page — there is only one source of truth for urgency.

Q: Why did you choose Lucide icons instead of emoji or images?

A: Lucide icons are lightweight, scale cleanly at any size, and look identical across all operating systems and browsers, unlike emoji which can look different (or missing) depending on the device. They also give the app a more polished, professional appearance.

Q: How is the completion rate calculated?

A: It is simply the number of completed assignments divided by the total number of assignments, converted to a percentage. This number is then shown both as text and drawn visually as a circular progress ring using SVG.

Q: Does the app support multiple users?

A: No — because data is stored in localStorage, it is tied to one specific browser on one specific device. It is designed as a single-user personal tool, similar to a personal notebook rather than a shared platform.

Q: Why does a "toast" notification pop up sometimes?

A: When there are urgent (critical or overdue) assignments, the app shows a one-time pop-up alert per browser session to make sure the student notices, without being repeatedly annoying every time they refresh the page.

Q: Could this project be turned into a mobile app or use a real database later?

A: Yes. Because the logic is already cleanly separated in common.js, the same functions could be reconnected to a real backend (like Firebase or a Node.js server) later, replacing only the localStorage calls, without needing to rewrite the whole app.

8. Summary / Closing Pitch

DeadlineDesk is a fully working, offline, browser-based tool that helps students stay on top of every assignment by combining clear visual risk indicators, live countdowns, a calendar view, and a progress dashboard — all built from scratch using only HTML, CSS and JavaScript. It solves a real problem every student faces, needs no server, account or internet connection to work, and remembers everything through the browser's built-in localStorage. The project demonstrates practical front-end development skills: structuring multi-page sites, building a consistent design system with light and dark themes, and writing clean, reusable JavaScript logic. I am confident presenting it because every feature — from the risk colours to the calendar grid — is something I can explain and justify from the code itself.